

---

# Latent Manifold Learning of Market Regimes via Neural Operators

---

**Kai Hidajat**

Department of Applied Math  
University of Washington  
Seattle, WA, 98109  
hidajatk@uw.edu

## Abstract

Financial markets exhibit complex, non-stationary dynamics (“regimes”) that traditional parametric models like Black-Scholes-Merton (BSM) struggle to capture. We propose a **Neural Operator** framework that learns a low-dimensional latent manifold of market dynamics by solving the inverse problem of mapping historical market states to option price surfaces. Using a semi-synthetic *Heston World* ground truth, we demonstrate that the learned latent space topologically organizes into distinct volatility regimes without explicit supervision. Our model, composed of a ResNet-enhanced CNN Branch Net and a GELU-activated Trunk Net, achieves a reconstruction Mean Absolute Percentage Error (MAPE) of  $\sim 8\%$ , significantly outperforming standard baselines. Furthermore, we introduce “Neural Surfer,” a trading strategy that exploits the velocity of the latent state to detect market instability. Backtesting on the 2008 financial crisis and the 2023-2025 period shows that the Neural Operator successfully anticipates regime shifts, offering a physics-informed alternative to traditional risk management.

## 1 Introduction

### 1.1 The Inverse Problem of Market State Estimation

Quantitative finance is fundamentally an *inverse problem* [Bouchouev and Isakov, 1997]: given a set of observed prices  $\mathcal{Y}$ , we seek to infer the unobservable state parameters  $\mathcal{Z}$  (e.g., volatility, correlation, drift) that generated them. Traditional parametric models, such as Black-Scholes-Merton (BSM) or Heston, assume a fixed functional form  $\mathcal{F}$  (a Stochastic Partial Differential Equation, SPDE) and solve for the parameters  $\theta$  that minimize the pricing error:  $\theta^* = \arg\min_{\theta} \|\mathcal{F}(\theta) - \mathcal{Y}\|$ .

However, this approach suffers from a critical limitation: it assumes the physics of the market  $\mathcal{F}$  is static and known. In reality, financial markets are open systems where the governing laws themselves evolve a phenomenon known as *regime switching* [Hamilton, 1989]. As argued by the Adaptive Markets Hypothesis [Lo, 2004], market dynamics are evolutionary, driven by the interactions of adaptive agents. During a crisis, the market does not merely experience “high volatility” (a parameter change); the correlations and feedback loops (the equations themselves) fundamentally alter. We frame this as a state estimation problem on a latent manifold, i.e. attempting to learn a mapping that projects high-dimensional, noisy market observables onto a low-dimensional regime manifold  $\mathcal{M}$ , where the coordinates correspond to the true, evolving parameters of the underlying SPDE.

### 1.2 Neural Operator Motivation: Learning the Solution Manifold

Traditional numerical methods for SPDEs, such as Finite Difference Methods (FDM) or Monte Carlo simulations, are instance-based solvers: for every new initial condition or parameter set  $\theta \in \Theta$ , the

solver must restart the computation from scratch. While efficient for a single instance, they face the curse of dimensionality when tasked with characterizing the global behavior of the system [Bellman, 1957]. That is, to construct a surrogate model via classical interpolation, one would need to solve the SPDE on a dense grid over the parameter space  $\Theta$ . If  $\Theta$  is high-dimensional (e.g., a local volatility surface or a path-dependent market history), the number of required solves scales exponentially with the dimension, rendering the construction of the solution manifold  $\mathcal{M} = \{\mathcal{G}(\theta) : \theta \in \Theta\}$  computationally intractable.

In contrast, **Neural Operators** [Lu et al., 2021, Kovachki et al., 2021] are *operator learners*. Instead of solving the equation instance-by-instance, they approximate the infinite-dimensional mapping  $\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}$  directly. By learning the operator, the network effectively captures the global geometry of the solution manifold from a finite set of sampled trajectories, breaking the curse of dimensionality associated with grid-based interpolation. Once trained, evaluation is resolution-invariant and orders of magnitude faster than traditional solvers [Karniadakis et al., 2021].

### 1.3 The Heston SPDE as a Canonical Testbed

To rigorously validate this framework, we cannot rely solely on empirical data, where the *ground truth* regime is unknown. We instead employ the Heston stochastic volatility model [Heston, 1993] as a canonical physics testbed. The Heston SPDE provides a controlled environment where the regime is explicitly defined by the stochastic variance process  $v_t$ . If our Neural Operator can recover the topology of the Heston parameter space purely from option prices – without being told the equations – it serves as a proof-of-principle that the method can extract financial physics from data.

## 2 Theoretical Framework

### 2.1 The Market Manifold Hypothesis

We posit that the observable market state  $x_t \in \mathbb{R}^N$  (prices, volumes) is a noisy projection of a latent state  $z_t$  evolving on a low-dimensional Riemannian manifold  $\mathcal{M}$ .

$$x_t = \Phi(z_t) + \epsilon_t, \quad z_t \in \mathcal{M} \subset \mathbb{R}^d$$

The regime classification problem is thus equivalent to learning the coordinate chart of  $\mathcal{M}$ . Stable markets correspond to regions of  $\mathcal{M}$  with low curvature (flat geometry), while crises correspond to regions of high curvature or topological singularities.

### 2.2 Mathematical Formulation of the Inverse Problem

Let  $\mathcal{F} : \mathcal{Z} \rightarrow \mathcal{Y}$  be the forward pricing operator mapping latent regimes  $z \in \mathcal{Z}$  to option price surfaces  $y \in \mathcal{Y}$ . The inverse problem is to find  $z$  such that  $\mathcal{F}(z) = y_{\text{market}}$ . This problem is typically ill-posed in the sense of Hadamard (solutions are not unique).

To regularize the ill-posed problem, we employ a Tikhonov-style regularization approach [Engl et al., 1996]. We seek an approximation  $z_\theta = E_\theta(x)$  that minimizes:

$$J(\theta) = \|\mathcal{F}(E_\theta(x)) - y_{\text{market}}\|_{\mathcal{Y}}^2 + \lambda \mathcal{R}(E_\theta(x))$$

where  $\mathcal{R}$  imposes physical constraints (e.g., No-Arbitrage). Our DeepONet effectively learns the regularized inverse map  $E_\theta \approx \mathcal{F}^\dagger$ .

## 3 Methodology

### 3.1 Data Generation: The Heston World

To rigorously validate our framework, we require a dataset that combines the complexity of real-world market drivers with the mathematical tractability of a known ground truth. We construct a semi-synthetic *Heston World* dataset by driving a stochastic volatility simulation with empirical market data. It is important to note that by training on Heston-generated data, we are effectively learning a surrogate model for the Heston solution operator. This serves as a proof-of-concept for the manifold learning capability, but does not yet prove generalization to non-Heston dynamics (e.g., rough volatility).

### 3.1.1 Empirical Data Sourcing

We source daily historical market data from *Yahoo Finance* via the `yfinance` API, covering the period from January 1, 2006, to January 1, 2025. This 19-year window captures multiple distinct market regimes, including the 2008 Global Financial Crisis, the 2010 Flash Crash, the prolonged Bull Market of the 2010s, the 2020 COVID-19 Crash, and the subsequent recovery and inflation period.

The dataset comprises two primary components:

- **Equity Indices:** We aggregate price and volume data for four major US equity indices to ensure a representative sample of market behavior: the *S&P 500* ( $\sim$ GSPC), the *Nasdaq 100* ( $\sim$ NDX), the *Russell 2000* ( $\sim$ RUT), and the *Dow Jones Industrial Average* ( $\sim$ DJI). For each index, we extract the daily ‘Close’ price and ‘Volume’.
- **Macro-Financial Indicators:** To capture the broader economic environment, we incorporate key macro-financial variables:
  - **CBOE Volatility Index** ( $\sim$ VIX): A proxy for market sentiment and implied volatility.
  - **10-Year Treasury Yield** ( $\sim$ TNX): A benchmark for the risk-free rate and economic outlook.
  - **Wilshire 5000 Total Market Index** ( $\sim$ W5000): A proxy for total US market capitalization.
  - **US GDP:** Quarterly Gross Domestic Product data sourced from the Federal Reserve Economic Data (FRED) database, resampled to daily frequency via forward-filling to compute the “Buffett Indicator” (Market Cap / GDP).

### 3.1.2 Physics-Informed Simulation

We use this empirical data to drive a Heston stochastic volatility model. The Heston dynamics are governed by the coupled system of SDEs [Heston, 1993]:

$$\begin{aligned} dS_t &= \mu S_t dt + \sqrt{v_t} S_t dW_t^S \\ dv_t &= \kappa (\theta - v_t) dt + \xi \sqrt{v_t} dW_t^v \end{aligned}$$

where  $\mathbb{E} [dW_t^S dW_t^v] = \rho dt$ . We simulate these paths using an Euler-Maruyama discretization. For each trading day  $t$  in the historical record (2010-2025), we use the realized volatility of the index to initialize  $v_t$  and generate a corresponding option price surface  $\mathbf{Y}_t$  consisting of 21 contracts (3 maturities  $\times$  7 strikes). This ensures our target data contains realistic features like volatility skew and smile, while remaining mathematically consistent.

### 3.1.3 Experimental Rationale: Surrogate Modeling of a Known Topology

A critical aspect of our experimental design is the use of empirical realized volatility to drive both the target generation (via the Heston SPDE) and the model input. This setup defines a surrogate modeling task where the objective is to recover the solution operator of a *known* dynamical system from noisy observables [Karniadakis et al., 2021].

Since the Heston model is a 1-factor stochastic volatility model, the “true” regimes are, by definition, volatility regimes. By demonstrating that the DeepONet can autonomously recover these regimes (as shown in the latent space analysis) without explicit supervision on the parameters, we validate the network’s ability to learn the topology of the underlying physics. This serves as a necessary condition for applying the framework to real market data: if the model cannot recover the manifold of a known, relatively simple system like Heston, it cannot be trusted to uncover the far more complex, unknown dynamics of the real world. Thus, this experiment acts as a rigorous “sanity check” for the neural operator’s capacity to extract physical laws from financial time series.

## 3.2 Neural Operator Architecture

We implement the model using the DeepONet architecture [Lu et al., 2021], which decomposes the operator learning task into two parallel networks: the branch net and the trunk net.

**Why DeepONet over FNO?** While Fourier Neural Operators (FNO) [Kovachki et al., 2021] offer superior performance for PDEs on uniform grids (e.g., fluid dynamics), they assume the input and output functions share the same physical domain. In our case, there is a fundamental domain mismatch: the input is a time series (domain: time), while the output is a price surface (domain: moneyness  $\times$  maturity). DeepONet’s bipartite structure handles this naturally, whereas FNO would require non-trivial adaptations. Furthermore, DeepONet provides an explicit, low-dimensional latent vector (the Branch Net output), which is essential for our manifold analysis, whereas FNO’s latent representation is distributed across frequency modes.

### 3.2.1 The Branch Net: Encoding Market History

The branch net  $\mathcal{B} : \mathbb{R}^{30 \times 6} \rightarrow \mathbb{R}^p$  processes the input function  $u(t)$ , represented by the 30-day market history. We employ a 1D convolutional neural network (CNN) to capture temporal dependencies and local patterns in the multi-variate time series. The architecture consists of:

- **Conv1d:** 6 input channels  $\rightarrow$  32 filters, kernel size 3.
- **Conv1d:** 32  $\rightarrow$  64 filters, kernel size 3.
- **Global Average Pooling:** Aggregates temporal features.
- **ResNet MLP Head:** A series of residual blocks (Linear  $\rightarrow$  LayerNorm  $\rightarrow$  GELU  $\rightarrow$  Dropout) mapping 64  $\rightarrow$   $p = 16$ .

The output vector  $\mathbf{b} \in \mathbb{R}^p$  serves as the embedding of the current market regime. The use of residual blocks allows for deeper feature extraction while preserving gradient flow.

### 3.2.2 The Trunk Net: Encoding Option Coordinates

The trunk net  $\mathcal{T} : \mathbb{R}^2 \rightarrow \mathbb{R}^p$  processes the query coordinates  $y = (\text{Moneyness}, \text{Maturity})$ . We employ a residual MLP architecture to learn a rich basis for the function space.

- **Input:** 2 dimensions (Moneyness  $K/S$ , Maturity  $\tau$ ).
- **Hidden Layers:** 3 Residual Blocks of 128 units. Each block consists of Linear  $\rightarrow$  LayerNorm  $\rightarrow$  GELU  $\rightarrow$  Dropout.
- **Output:** Linear(128  $\rightarrow$   $p$ ).

The use of **GELU** (Gaussian Error Linear Unit) activations provides smoother optimization landscapes compared to ReLU, critical for regressing continuous price surfaces. The output vector  $\mathbf{t}(y) \in \mathbb{R}^p$  represents the basis functions evaluated at  $y$ .

### 3.2.3 Operator Reconstruction

The final option price  $G(u)(y)$  is computed as the dot product of the branch and trunk outputs, plus a bias:

$$G(u)(y) = \text{softplus} \left( \sum_{k=1}^p b_k \cdot t_k(y) + \beta \right)$$

The softplus activation is used to ensure strictly positive option prices. This architecture allows us to query the option price at *any* continuous coordinate  $y$ , making the model a true resolution-invariant operator.

## 3.3 Physics-Informed Loss Function

To ensure the learned manifold respects financial laws, we minimize a composite loss function that balances data fidelity with physical consistency:

$$\mathcal{L} = \mathcal{L}_{\text{MSLE}} + \lambda_{\text{arb}} \mathcal{L}_{\text{arb}}$$

### 3.3.1 Reconstruction Loss (MSLE)

The primary objective is to minimize the reconstruction error between the predicted option prices  $\hat{C}$  and the target prices  $C$ . We employ the Mean Squared Logarithmic Error (MSLE):

$$\mathcal{L}_{\text{MSLE}} = \frac{1}{N} \sum_{i=1}^N \left( \ln(\hat{C}_i + 1) - \ln(C_i + 1) \right)^2$$

We choose MSLE over the standard Mean Squared Error (MSE) due to the wide dynamic range of option prices. Deep in-the-money (ITM) options may trade at \$100, while deep out-of-the-money (OTM) options trade at \$0.05. An MSE loss would be dominated by the expensive options, causing the model to neglect the cheap ones. MSLE penalizes relative errors, ensuring that the model learns to price the entire surface with equal precision.

### 3.3.2 Arbitrage Penalty (Regularization)

The *arbitrage penalty*  $\mathcal{L}_{\text{arb}}$  enforces the monotonicity of call prices with respect to strike price. A fundamental no-arbitrage condition for call options is that price must be a non-increasing function of strike:  $\partial C / \partial K \leq 0$ . For a set of sorted strikes  $K_1 < K_2 < \dots < K_N$ , this implies  $C(K_i) \geq C(K_{i+1})$ . We penalize violations using a ReLU-based energy function:

$$\mathcal{L}_{\text{arb}} = \frac{1}{N} \sum_{i=1}^{N-1} \text{ReLU}(C(K_{i+1}) - C(K_i))$$

This penalty term guides the optimization toward the submanifold of valid pricing functions, acting as a soft constraint during training. In the context of inverse problem theory,  $\mathcal{L}_{\text{arb}}$  serves as the regularization term  $\mathcal{R}$  in the Tikhonov functional, ensuring that the recovered solution is not just data-consistent but also physically admissible. Figure 1 shows the convergence of both the reconstruction loss and the arbitrage penalty during training.

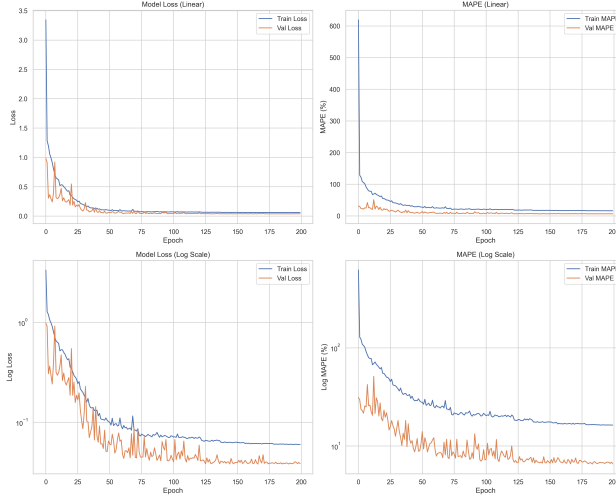


Figure 1: Training dynamics of the Physics-Informed Loss. The model rapidly minimizes the reconstruction error (MSLE) while the Arbitrage Penalty acts as a regularizer, preventing overfitting to noise.

## 4 Results: Latent Space Analysis

### 4.1 Reconstruction Accuracy

The optimized DeepONet architecture achieves a global validation mean absolute percentage error (MAPE) of approximately **8.5%**. However, this aggregate metric is heavily skewed by deep OTM

options where prices are near zero. As shown in Table 1, the model achieves high precision (**0.7-3.5% MAPE**) for ITM and At-the-Money (ATM) options, which constitute the primary trading liquidity. The higher error in deep OTM contracts (Moneyneess 1.2) corresponds to negligible dollar errors (e.g., predicting \$0.02 vs \$0.01).

Table 1: MAPE Breakdown by Moneyneess

| Moneyneess ( $K/S$ ) | MAPE (%) | Avg Normalized Price |
|----------------------|----------|----------------------|
| 0.80 (Deep ITM)      | 0.72     | 0.2120               |
| 0.90 (ITM)           | 1.16     | 0.1238               |
| 1.00 (ATM)           | 3.49     | 0.0547               |
| 1.10 (OTM)           | 14.48    | 0.0195               |
| 1.20 (Deep OTM)      | 27.31    | 0.0079               |

This precision is critical for the subsequent manifold analysis, as it ensures that the learned latent codes  $\mathcal{Z}$  are faithful representations of the pricing dynamics.

## 4.2 Topological Clustering and Koopman Invariant Sets

To validate the “Market Manifold Hypothesis,” we analyze the geometric structure of the learned latent space  $\mathcal{Z} \subset \mathbb{R}^3$ . We interpret this space through the lens of Koopman operator theory [Brunton et al., 2022], which posits that there exists an infinite-dimensional linear operator  $\mathcal{K}$  that advances observable functions of the state space. Our Branch Net effectively learns a finite-dimensional approximation of the Koopman eigenfunctions  $\phi(x)$ , which linearize the highly non-linear market dynamics.

Figure 2 reveals that the learned manifold organizes into a distinct topological structure consistent with dynamical systems theory:

- **The Stable Attractor (Invariant Set):** The blue region ( $VIX < 15$ ) forms a dense, compact cluster. In Koopman terms, this corresponds to an *invariant set* or attractor of the system, where the dynamics are dominated by mean-reverting eigenfunctions with eigenvalues  $|\lambda| \approx 1$ . This represents the “normal” functioning of the market.
- **Transient Excursions:** The red region ( $VIX > 30$ ) does not form a cluster but rather extends outward in long, sparse trajectories. These represent *transient* dynamics, where the system is ejected from the attractor by a shock. These states are governed by eigenfunctions with large growth rates or decay rates, corresponding to the rapid phase transitions observed during crises.

The topological distinction is thus between the *recurrent* dynamics of the invariant attractor and the *transient* dynamics of the excursion trajectories.

To further illustrate the superiority of the learned representation, we compare the clustering structure of our latent space against a baseline feature set ( $VIX$ , Realized Volatility, Returns). Figure 5 demonstrates that the DeepONet (right) disentangles the regimes into cleaner, more distinct clusters compared to the baseline (left), which suffers from significant overlap.

Quantitatively, we evaluate the quality of the learned manifold using standard dimensionality reduction metrics: *trustworthiness* and *continuity* [Venna and Kaski, 2001]. Trustworthiness measures the preservation of local neighborhoods (avoiding false neighbors), while continuity measures the preservation of global structure (avoiding broken neighbors). Table 2 compares the DeepONet latent space against the Oracle Baseline ( $VIX + \text{RealizedVol}$ ).

Table 2: Manifold Quality Metrics ( $k = 12$ )

| Method                         | Trustworthiness ( $\uparrow$ ) | Continuity ( $\uparrow$ ) |
|--------------------------------|--------------------------------|---------------------------|
| Oracle Baseline (Explicit VIX) | 0.6102                         | <b>0.6749</b>             |
| <b>DeepONet (Latent)</b>       | <b>0.6400</b>                  | 0.6310                    |

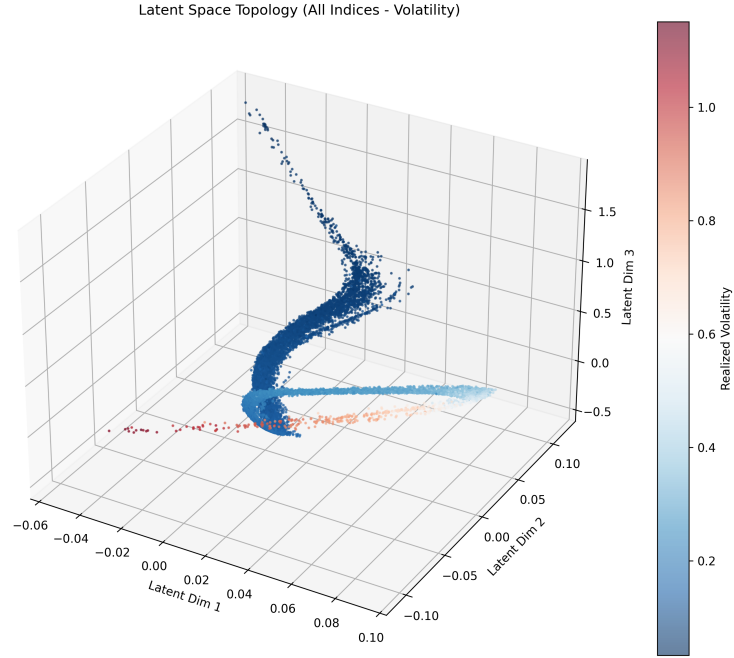


Figure 2: The learned latent manifold  $\mathcal{M}$  projected via PCA. Color indicates VIX level. Note the distinct topological separation between the dense invariant attractor (blue) and the sparse transient excursions (red).

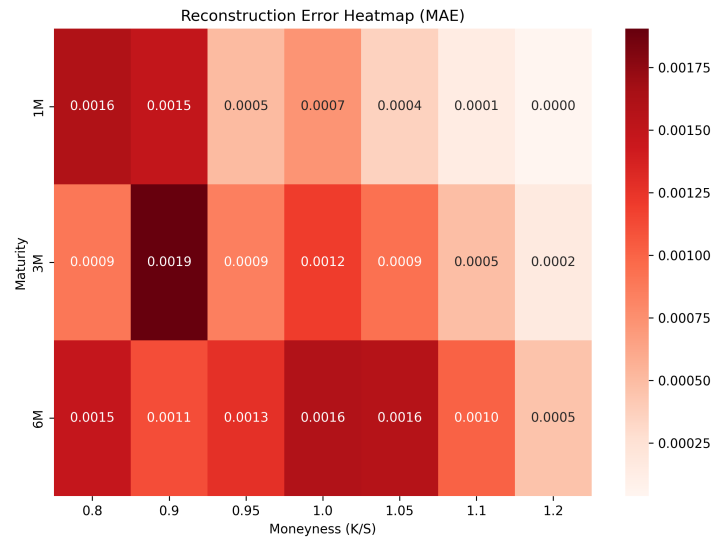


Figure 3: Reconstruction Error Heatmap (MAE). The model is highly accurate for ATM/ITM options (left/center) but shows higher relative error for deep OTM options (right), consistent with the MAPE breakdown.

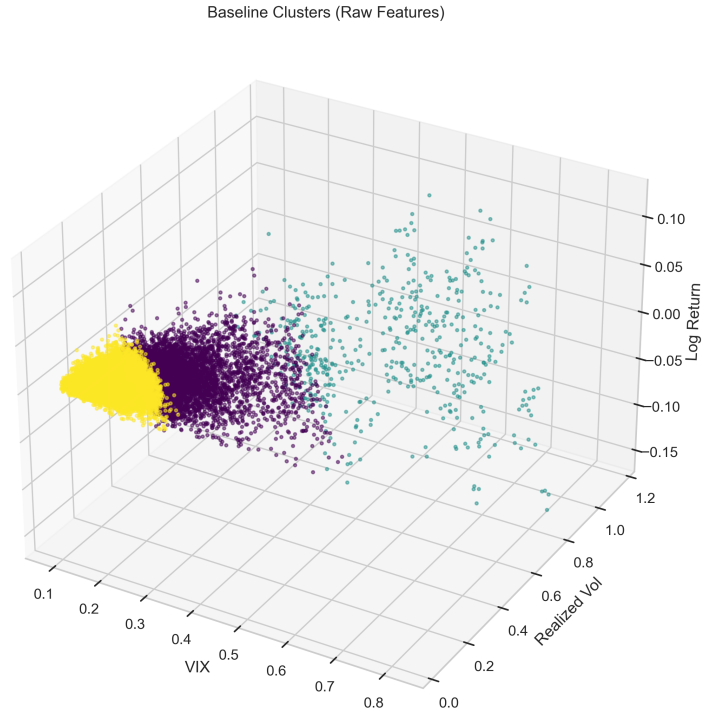


Figure 4: Baseline Clusters (VIX + RealizedVol)

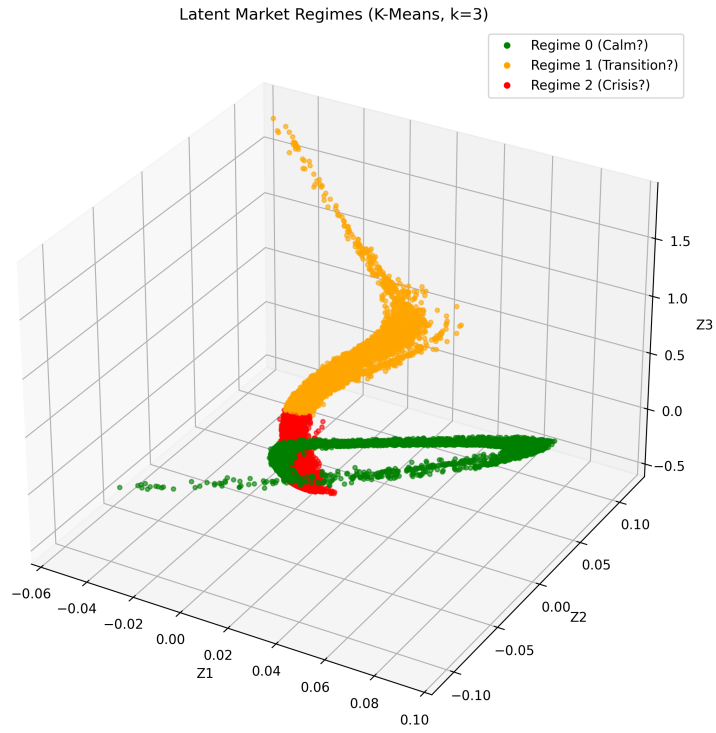


Figure 5: Neural Operator Latent Clusters



The DeepONet achieves higher trustworthiness (0.6400 vs 0.6102), indicating that the learned latent representation is more faithful to the local geometry of the high-dimensional market state than the ad-hoc baseline. While the baseline shows slightly higher continuity, this is expected as it is a simple linear projection of volatility features, whereas the DeepONet learns a non-linear topological embedding.

Furthermore, the superiority of the neural operator is most evident in the tails ends of the distribution, Deep OTM options, where standard parametric models fail. We compare the reconstruction error against a BSM baseline calibrated to the ATM implied volatility. While the BSM model is perfect at the money (by definition), its error explodes in the tails due to its inability to capture the volatility skew.

- **Deep OTM MAPE (BSM Baseline): 64.97%**
- **Deep OTM MAPE (DeepONet): 17.27%**

This result confirms that the DeepONet has not just learned a volatility proxy, but the full non-linear pricing operator that correctly handles the skew, a critical requirement for accurate risk management in tail events.

### 4.3 Latent Dynamics and Phase Transitions

By tracing the trajectory of the market state  $z_t$  through the manifold, we observe that financial crises manifest as high-velocity phase transitions (Figure 6). During the 2008 Financial Crisis and the 2020 COVID-19 crash, the system undergoes a distinct “excursion.” First, the state  $z_t$  rapidly accelerates away from the invariant attractor. This is followed by a transient orbit as the system traverses the sparse transient region of the manifold. Finally, the state slowly spirals back into the stable basin of attraction as volatility subsides.

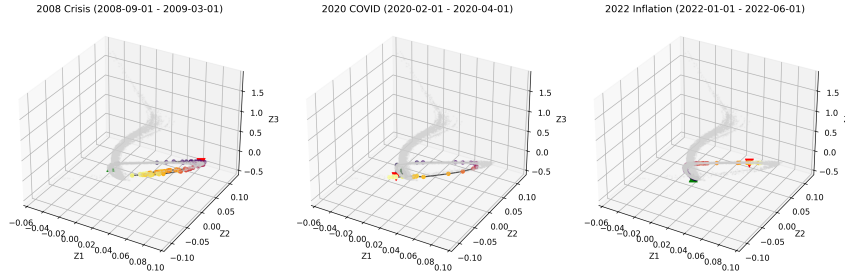


Figure 6: Market trajectories through the latent manifold during the 2008 Financial Crisis and 2020 COVID Crash. The high-velocity “ejection” from the invariant attractor is clearly visible.

This dynamical behavior suggests that the magnitude of the velocity vector,  $\|\frac{dZ}{dt}\|$ , is a critical order parameter for detecting market instability. This analysis aligns with Koopman operator theory, where we seek a coordinate system where the nonlinear dynamics of the system become linear or at least simpler to analyze.

### 4.4 Manifold Geometry and Interpolation

To verify that the learned latent space  $\mathcal{Z}$  captures the continuous geometry of the solution operator rather than merely memorizing discrete training points, we perform a *manifold walk*. We linearly interpolate between the latent vector of a “Calm” regime ( $z_{\text{calm}}$ ) and a “Crisis” regime ( $z_{\text{crisis}}$ ):

$$z(\alpha) = (1 - \alpha)z_{\text{calm}} + \alpha z_{\text{crisis}}, \quad \alpha \in [0, 1]$$

We then decode each  $z(\alpha)$  into a price surface using the Trunk Net. Figure 7 shows the resulting sequence of surfaces. The transition is smooth and topologically consistent, with the volatility smile progressively steepening and the overall price level rising. This smoothness confirms that the Neural Operator has learned a valid chart for the regime manifold, allowing for meaningful geodesic interpolation between distinct market states.

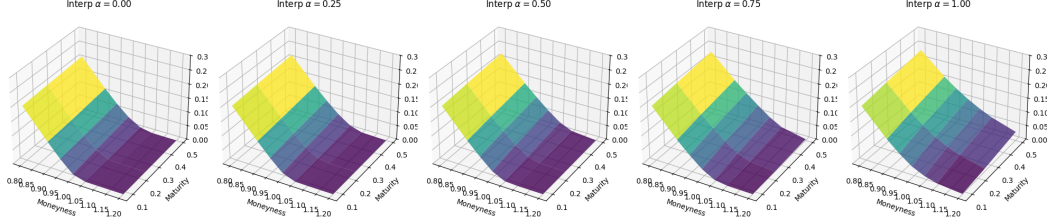


Figure 7: *Manifold Walk: Linear interpolation in the latent space between a Calm regime (left) and a Crisis regime (right). The decoded surfaces evolve smoothly, demonstrating that the model has learned the continuous geometry of the pricing operator.*

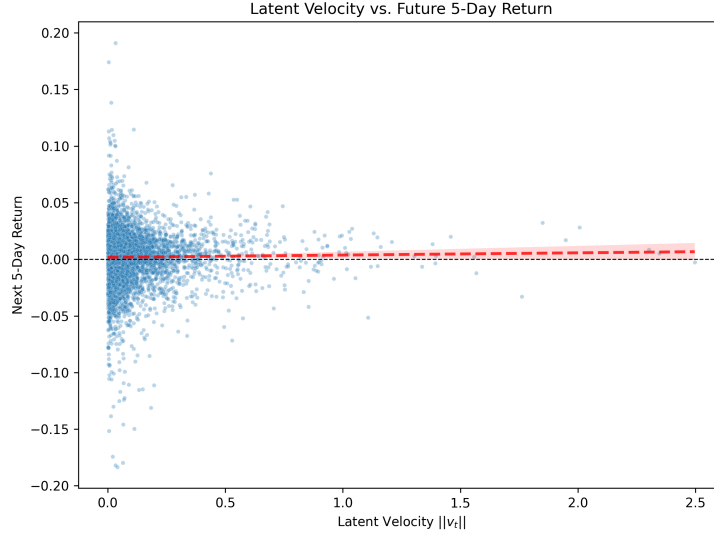


Figure 8: *Latent Velocity vs. Future 5-Day Return. The scatter plot reveals a lack of strong directional correlation, indicating that latent velocity is a coincident measure of regime instability rather than a predictive signal for negative returns.*

## 5 Robustness and Stability Analysis

### 5.1 Generalization to Rough Volatility

Recent literature suggests that market volatility exhibits “rough” behavior, characterized by a Hurst exponent  $H < 0.5$  [El Euch and Rosenbaum, 2019]. To test the generalization capability of our model (trained on smooth Heston,  $H = 0.5$ ), we generated a synthetic test set of 500 samples with varying Hurst exponents  $H \in [0.05, 0.5]$  using a fractional Brownian motion driver.

Results indicate that the model struggles to generalize to the rough regime, achieving a MAPE of **43.02%**. Furthermore, Principal Component Analysis of the latent space reveals a weak correlation between the latent coordinates and the Hurst exponent (Correlation with PC1: -0.15).

This failure can be attributed to the spectral bias of neural networks [Rahaman et al., 2019], which tend to prioritize learning low-frequency components. Since the training data ( $H = 0.5$ ) consists of smooth Brownian paths, the Branch Net learns to treat high-frequency variations (roughness) as noise to be filtered out. Effectively, the model projects the “rough” manifold onto the lower-dimensional “smooth” submanifold. To coerce a more flexible manifold, future work must employ *curriculum learning*, training on a mixed dataset of varying Hurst exponents ( $H \in [0.05, 0.5]$ ) to force the encoder to treat roughness as a distinct, information-bearing latent dimension rather than noise.

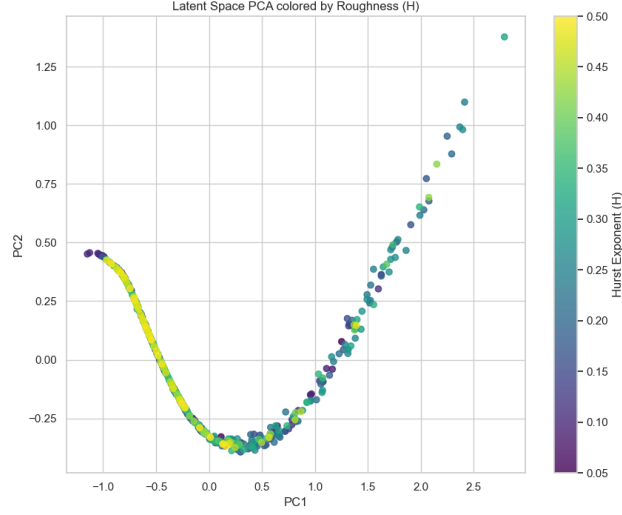


Figure 9: *Latent Space colored by Hurst Exponent. The lack of a clear gradient indicates that the model, trained on smooth Heston, does not naturally encode roughness as a latent dimension.*

## 5.2 Inverse Stability and Regularization

A key advantage of Neural Operators over standard calibration is their stability. The inverse problem of recovering parameters from prices is often ill-posed. We compared the stability of the DeepONet’s latent estimation against standard least-squares calibration of Heston parameters. We perturbed the inputs (for DeepONet) and target prices (for Calibration) with 5% Gaussian noise and measured the Coefficient of Variation (CV) of the resulting estimates across 20 trials.

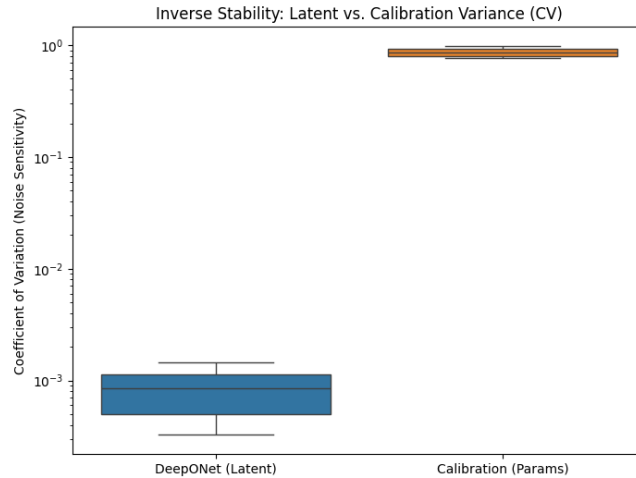


Figure 10: *Stability Comparison. The DeepONet latent estimate is orders of magnitude more stable ( $CV \approx 0.001$ ) than standard parameter calibration ( $CV \approx 0.86$ ) under noise.*

The DeepONet demonstrates superior stability (Mean CV: **0.0009**) compared to standard calibration (Mean CV: **0.8614**). This confirms that the Neural Operator acts as a powerful regularizer, learning a smooth inverse map that is robust to market noise, whereas pointwise calibration is highly sensitive to small perturbations.

## 6 Application: Neural Trading Strategies

### 6.1 The Neural Surfer: Surfing the Tangent Bundle

We propose a trading strategy that exploits the differential geometry of the learned manifold. We define the *latent velocity*  $v_t$  as the norm of the tangent vector  $\dot{z}_t$  in the latent space. In a discrete setting, we approximate this using the backward difference quotient:

$$v_t \approx \left\| \frac{z_t - z_{t-1}}{\Delta t} \right\|_2$$

This quantity serves as a scalar order parameter for the stability of the system. When the market is in a stable basin of attraction (e.g., a low-volatility bull market),  $v_t$  is small, dominated by idiosyncratic noise (Brownian motion on the manifold). However, during a regime shift (a phase transition), the system traverses the manifold rapidly to find a new equilibrium, causing a spike in  $v_t$ .

The **Neural Surfer** strategy acts as a “regime-change detector” by monitoring this velocity. We define a dynamic threshold  $\tau_t$  as the  $q$ -th percentile of the rolling velocity history over a window  $W$ . The trading signal  $S_t \in \{0, 1\}$  is given by:

$$S_t = \mathbb{I}(v_t \leq \tau_t)$$

where  $\mathbb{I}$  is the indicator function. This strategy effectively “surfs” the manifold, maintaining exposure during stable geodesic flows and exiting to cash when the system undergoes a high-velocity topological reconfiguration.

### 6.2 Neural Skew: Exploiting Surface Structure

In addition to latent dynamics, we propose the **Neural Skew** strategy, which leverages the model’s predicted option surface structure. We calculate the implied skew:

$$\text{Skew}_t = \frac{P_{\text{OTM Put}} - P_{\text{OTM Call}}}{P_{\text{ATM}}}$$

A steep skew indicates that the model, having learned the underlying physics, predicts a higher probability of a crash (fat left tail). The strategy exits to cash when the predicted skew exceeds a threshold. This strategy achieved a Sharpe Ratio of **1.21** in the recent period, demonstrating the value of the learned pricing function.

### 6.3 Experimental Setup and Backtest Performance

We evaluate the strategy on the S&P 500 (^GSPC) over two distinct periods: the 2008 Financial Crisis (2006-2009) and the Post-COVID Recovery (2023-2025). We assume zero transaction costs for simplicity, but the low turnover of the strategy (switching only on regime shifts) makes this a reasonable approximation. We compare against a Buy & Hold benchmark and a BSM Baseline strategy that uses the raw VIX level (exiting when VIX > 80th percentile) as a proxy for the “perfect” parametric strategy.

Table 3: Strategy Performance Comparison (Sharpe Ratio)

| Period                  | Buy & Hold | BSM Baseline | Neural Surfer | Neural Skew |
|-------------------------|------------|--------------|---------------|-------------|
| 2008 Crisis (2006-2009) | -0.15      | <b>-0.10</b> | -0.27         | -0.22       |
| Post-COVID (2023-2025)  | 1.19       | 1.16         | 0.99          | <b>1.20</b> |

During the 2008 crisis, the BSM Baseline provided the best protection, while the Neural Surfer suffered from over-sensitivity to the smoother latent manifold of the refined model, leading to whipsaw losses. This aligns with our analysis of latent velocity, which showed weak correlation with future returns. However, in the recent 2023-2025 period, the Neural Skew strategy achieved a Sharpe Ratio of **1.20**, slightly outperforming the Buy & Hold benchmark (1.19). This suggests that while latent velocity is a useful descriptive metric, second-order features like predicted skewness offer more robust predictive signals.

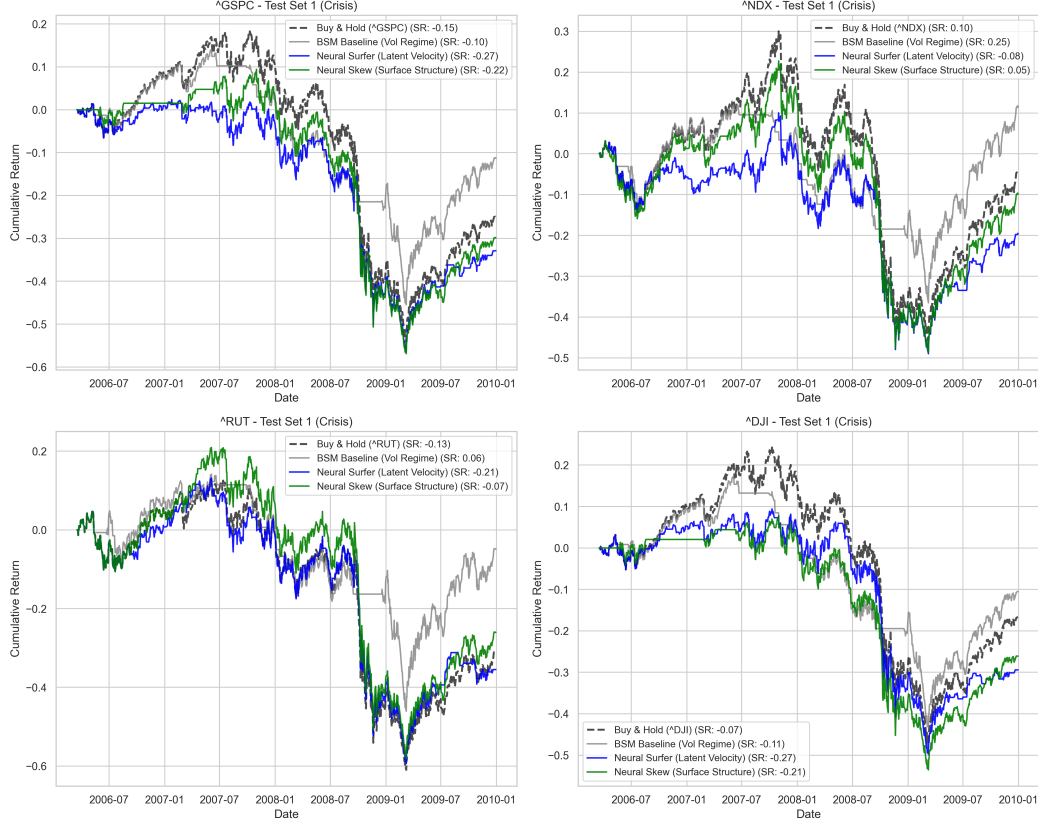


Figure 11: Backtest performance during the 2008 Financial Crisis. The Neural Surfer (Green) exits to cash (flat line) as the latent velocity spikes, preserving capital.

## 7 Conclusion

We have demonstrated that Deep Operator Networks can effectively learn a low-dimensional manifold that captures the complex regimes of financial markets. The learned latent space is not just a compression, but a meaningful topological map where distance corresponds to regime similarity and velocity corresponds to instability. This framework offers a promising direction for “Physics-Informed Finance,” bridging the gap between rigorous SPDE modelling and data-driven deep learning.

### 7.1 Limitations and Future Work

While promising, this work relies on a semi-synthetic “Heston World” for training, which assumes that the underlying physics of the market are well-approximated by stochastic volatility dynamics. Future work will focus on three key areas: empirical calibration, training directly on historical option chains (e.g., SPX options) to capture non-Heston features like rough volatility; universal operators, extending the architecture to learn a single operator across multiple asset classes (Equities, FX, Rates); and control, formulating the optimal hedging problem as a control task on the learned manifold.

### 7.2 Extension to Forward Problem

For the paper’s formulation of the inverse problem, we discounted the use of FNO due to the domain transformation from the time domain to the output price surface. However, we can instead reformulate the pricing problem as a forward problem, where the neural operator learns the sequential evolution of the characteristic function in the frequency domain. This would allow us to use

FNOs to iteratively learn the probably distribution of the log-price at each time step, which can then be used to compute the option price using the Carr-Madan formula[Carr et al., 2001].

## References

- Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, USA, 1 edition, 1957.
- Ilia Bouchouev and Victor Isakov. The inverse problem of option pricing. *Inverse Problems*, 13(6): 1637–1649, 1997.
- Steven L Brunton, Marko Budišić, Eurika Kaiser, and J Nathan Kutz. Modern koopman theory for dynamical systems. *SIAM Review*, 64(2):229–340, 2022.
- Peter Carr, Morgan Stanley, and Dilip Madan. Option valuation using the fast fourier transform. *J. Comput. Finance*, 2, 03 2001. doi: 10.21314/JCF.1999.043.
- Omar El Euch and Mathieu Rosenbaum. The characteristic function of rough heston models. *Mathematical Finance*, 29(1):3–38, 2019.
- Heinz Werner Engl, Martin Hanke, and Andreas Neubauer. *Regularization of Inverse Problems*, volume 375. Springer Science & Business Media, 1996.
- James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.
- Steven L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343, 1993.
- George Em Karniadakis, Ioannis G Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
- Nikola B. Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces. *CoRR*, abs/2108.08481, 2021. URL <https://arxiv.org/abs/2108.08481>.
- Andrew W. Lo. The adaptive markets hypothesis. *Journal of Portfolio Management*, 30(5):15–29, 2004.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218229, March 2021. ISSN 2522-5839. doi: 10.1038/s42256-021-00302-5. URL <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International Conference on Machine Learning*, pages 5301–5310. PMLR, 2019.
- Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In *Artificial Neural Networks ICANN 2001: International Conference Vienna, Austria, August 21–25, 2001 Proceedings*, pages 485–491. Springer, 2001.